

Completato Concetto di Ambiente

fino ad Ora

%% EXP + ID Costanti

%% DICHIARAZIONI + ID Costanti

} ⇒ AMBIENTE

Associazione ID ↔ Oggetto Denotato

SEMANTICA STATICA

CHE VERIFICA i vincoli

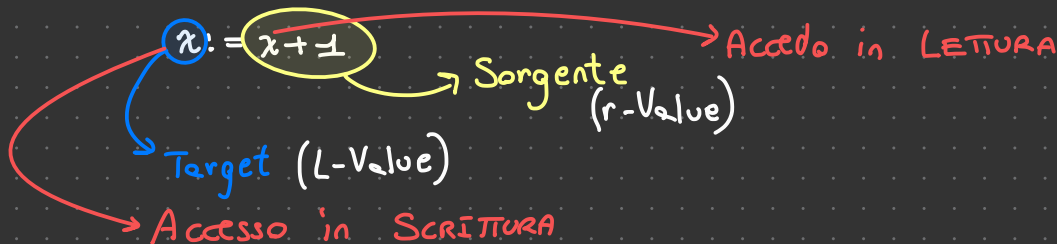
Concetto di TIPO (dell'Oggetto Denotato)

CONTESTUALE
(vincoli di TIPO)

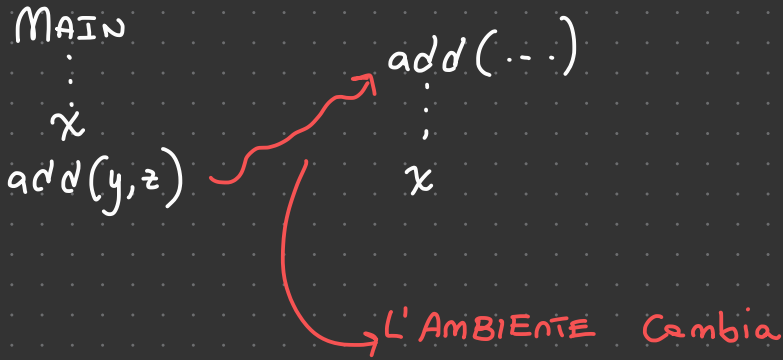
Per Arrivare ai COMANDI Bisogna Aggiungere la MEMORIA

MEMORIA:

Capire qual'è il Ruolo dell'ID Nell'Assegnamento



Il problema a dx è che a destra può Avere
Significati diversi:



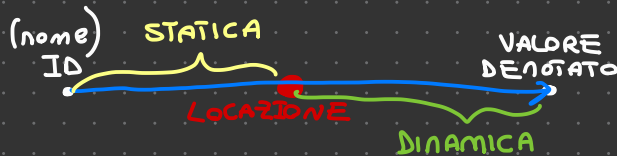
Cosa Può Contenere x ?
Che Valore Contiene x ? } Sono 2 cose ben diverse

Aliasing $\Rightarrow x$ e y possano riferirsi alla stessa cella di memoria

Mi serve allora un Altro Tipo di Associazione, ad ogni ID un Oggetto Denotato e anche il concetto di Localizzazione

LOCALIZIONE:

Concetto Astratto della cella di memoria (a livello logico la Rappresenta)



Ho 2 Parti: **STATICA** $\rightarrow$ tipo (nell' **AMBIENTE**)
DINAMICA $\rightarrow$ Valore Contenuto (Nella **Mem.**)

MEMORIA → Insieme di Associazioni Tra Locazioni e Valori Memorizzabili (non più Denotabili)

VALORI:

- Esprimibile se RAPPRESENTABILE Mediante EXP
- Denotabile (con Ambienti) sono Riferibili con ID, Posso dargli un Nome
- Memorizzabile se Possono Essere Associate a locaz.

La MEM. Serve per Catturare le azioni IREVERSIBILI generate dal PROGRAMMA

L'Ambiente Cattura Solo Trasformazioni REVERSIBILI

$$\left. \begin{array}{l} x := x + 1 \\ x := x - 1 \end{array} \right\} \begin{array}{l} \text{Ho Neutralizzato la prima Operazione,} \\ \text{facendo 2 Operazioni DIVERSE e SUCCESSIVE} \end{array}$$

x SOSPESA

→ DELIMITATORE DELL' AMBIENTE

{ int x CREAZIONE DI x

⋮

}

Tutte le Modifiche ad x Sono Annullate e x Stessa è ELIMINATA (quella Interna) e Viene Recuperato x

MEMORIA:

MEM, insieme di Memorie sono Associazioni Tra Locazione e Valore MEMORIZZABILE

Loc, insieme di Locazioni

MVal, insieme di Valori Memorizzabili

$$\sigma \in \text{MEM} \quad \sigma: \text{Loc} \rightarrow \text{Val}$$

$$L \subseteq_f \text{Loc} \quad \forall \sigma \in L. \quad \sigma: L \mapsto v \in \text{MVal}$$

$$\text{MEM}_L = L \rightarrow \text{MVal} = \text{MVal}$$

$$\text{MEM} = \bigcup_{L \subseteq_f \text{Loc}} \text{MEM}_L \quad \text{Insieme di Associazioni da un Qualunque}$$

$L \subseteq_f \text{Loc}$ $\rightarrow$ SOTTOINSIEME FINITO

Insieme finito di Locazioni ad un Insieme di Valori

NVal = Eval $\cup$ Loc che nel nostro linguaggio:

$$\text{NVal} = \text{N} \cup \text{B} \cup \text{Loc}$$

$$\text{e MVal} = \text{N} \cup \text{B}$$

AGGIORNAMENTO MEM

$\delta[\delta']$

$\delta, \delta' \in \text{MEM}$

$\delta: L$

$\delta': L'$

dominio di δ

dominio di δ'

$\delta'' = \delta[\delta'] \in \text{MEM}$

$$\forall l. l \in L \cup L' . \delta''(l) = \begin{cases} \delta'(l) & \text{con } l \in L' \\ \delta(l) & \text{con } l \in L \setminus L' \end{cases}$$

Aggiungiamo Alle dichiarazioni, la DICHIARAZIONE DI ID VARIABLE

Tipi PRIMA = $\{ \text{int}, \text{Bool} \}$

Dec: $D \rightarrow \dots \mid \text{Var } x: \tau = e$

tutte quelle già viste prima

Tipi ORA = $\{ \text{int}, \text{Bool}, \text{int}_{\text{loc}}, \text{Bool}_{\text{loc}} \}$

Sono MODIFICABILI

Devo CREARE un TIPO 'locazione' = τ_{loc} per dire che è una LOCAZIONE di TIPO τ

$\text{int}_{\text{loc}}, \text{Bool}_{\text{loc}} \Rightarrow$ Locazione Per INTERI, BOOLEANI

Rimane TIPO τ Quando Mettiamo CONST

SEMANTICA STATICA: dichiarazioni

Associa un Ambiente Statico

$$\Delta \vdash (\text{var } x:\tau = e) = \{x\} \quad FI(\text{var } x:\tau = e) = FI(e)$$

Aggiungo Queste 2 Definizioni a Quelle già date

$$\frac{\Delta \vdash e:\tau}{\Delta \vdash x:\tau = e : [x \mapsto \tau_{Loc}]}$$

SEMANTICA STATICA: Espressioni

$$\Delta \vdash I:\tau \text{ se } \Delta(I) \in \{\tau \text{ oppure } \tau_{Loc}\}$$

SEMANTICA DINAMICA: Cambia per Aggiunta delle MEM.

$$T: E \times MEM \rightarrow Val \times MEM$$

$\downarrow$
 $N \cup B$

Dove $T = V$ [Terminali Rimangono VALORI]

$$p \vdash \langle e, d \rangle \rightarrow \langle e', d \rangle$$

Espressioni

La SEMANTICA DINAMICA delle EXPR. Accede ALLA MEMORIA

in Cui Valutare l'ESPRESSIONE (Ma NON la modifica)

Nelle Dichiarazioni:

$$\mathcal{T}: D \times \text{Mem} \rightarrow \underbrace{\text{Env} \times \text{Mem}}_T$$

$$\rho \vdash \langle d, \delta \rangle \rightarrow \langle d', \delta' \rangle \quad \text{dove } \delta' \text{ può essere } \neq \delta$$

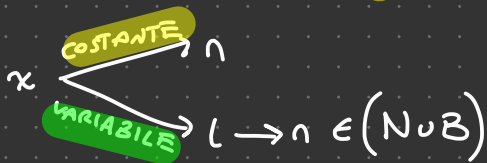
REGOLE DA AGGIUNGERE ALLE EXPR.:

$$\rho \vdash \langle x, \delta \rangle \rightarrow \langle n, \delta \rangle$$

$$\rho(x) = n$$

oppure

$$\rho(x) = l \underset{\text{LOC}}{\wedge} \text{ e } \delta(l) = n$$



REGOLE DA AGGIUNGERE ALLE DICHIARAZIONI:

$$\rho \vdash \langle e, \delta \rangle \xrightarrow{*} \langle k, \delta \rangle$$

$$\rho \vdash \langle \text{Var } x: r = e, \delta \rangle \rightarrow \langle [x: r = l], \delta[l \mapsto k] \rangle$$

$l \in \text{LOC}$

(nuova locazione)

Genera un **MODIFICA IRREVERSIBILE** con la MEMORIA e Questa è l'unica Regola che può Modificare la MEMORIA (che è un SIDE EFFECT, visto che Op. su MEM dovrebbero Essere

Solo dei Comandi)

Con Questa Inizializzazione delle Variabili Modifichiamo la MEMORIA

VALUTAZIONE DI UN EXPR. CON ID VARIABILI:

$Eval: Expr \times Mem \rightarrow Val$

$Eval(e, \delta) = (K, \delta)$ sse $\vdash \langle e, \delta \rangle \longrightarrow^* \langle K, \delta \rangle$

EQUIVALENZA: $\equiv \subseteq Expr \times Expr$

$\forall e_0, e_1 \in Expr. e_0 \equiv e_1$ se $\forall \delta. Eval(e_0, \delta) = Eval(e_1, \delta)$

ESEMPIO: $(2 + x - x)$ o Anche $(\frac{2x}{x})$ Sono

INVARIANTI ed INDIPENDENTI dal Valore Assegnato da x

ELABORAZIONE DI DICHIARAZIONI CON ID VAR:

$Elab: Dec \times Mem \rightarrow Env \times Mem$

$Elab(d, \delta) = (\rho, \delta')$ sse $\vdash \langle d, \delta \rangle \longrightarrow^* \langle \rho, \delta' \rangle$

EQUIVALENZA: deve Tener Conto dei SIDE EFFECTS (anche Sulla MEMORIA GENERATA)

$\forall d_1, d_2 \in Dec. d_1 \equiv d_2$ sse $\forall \delta \in Mem. Elab(d_1, \delta) = Elab(d_2, \delta)$

COMANDI:

Strumenti Per **MODIFICARE LA MEMORIA** (costrutti dedicati)

Nuova Categoria SINTATTICA, mi fanno Cambiare lo STATO DELLA MACCHINA.

Mi permettano di **DENOTARE** richieste di **MODIFICHE** alla mem.

Comandi **Vengono Eseguiti** per OTTENERE la TRASFORMAZIONE della MEMORIA DERIVATA

fino ad ORA Tutto è DECIDIBILE (Ma aggiungeremo il WHILE, con Problema della non TERMINAZIONE)

ASSIGNAMENTO → Base di Modifica della Memoria

COMPOSIZIONE SEQ. → Come Componiamo Pezzi del Progr.

CONTROLLI FLUSSO ESECUZIONE → Condizionale / Iterativo

SINTASSI:

$$C \rightarrow \text{Skip} \mid x := e \mid C; C \mid \text{WHILE } B \text{ do } C \mid \text{if } B \text{ Then } C \text{ Else } C$$

Elemento Neutro
Assegnamento (x è ID e EXP)

Composizione Sequenziale ⇒ Tutto il Primo Comando e Poi il Secondo

Ripete C fino a che B è Valutata Vero

Selta Condizionale $\Rightarrow$ Se B è Vero Esegua C Altrimenti
si Esegua il Ramo ELSE

Assegnamento Introduce il CONCETTO di Variabile che
Consiste in un ELEMENTO COMPLESSO in un PL

Elementi che Caratterizzano la Variabile:

/// Nome/ID

/// Tipo $\rightarrow$ Valori/Operazioni che Caratterizzano la Var.

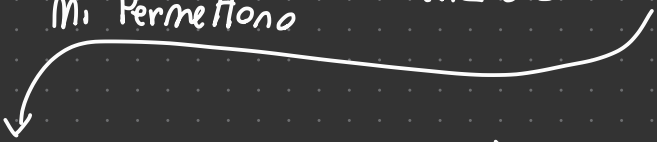
/// Valore $\rightarrow$ Denotabile/Locazione è un Val Riscribibile
Mediante l'occorrenza del Nome

/// Indirizzo $\rightarrow$ A Lvl. fisico è quello che Abbiamo astratto
con la LOCAZIONE

/// SCOPE $\rightarrow$ Visibilità nell' Ambiente / Raggio d'Azione ovvero
L'Ambiente di Riferimento

/// TEMPO DI VITA $\rightarrow$ Misura in Termini Temporeali dell'
Esistenza della Variabile (DIPENDE

Mi Permettono ANCHE DAL LINGUAGGIO)


DEALLOCAZIONE e Quindi diventa DINAMICO

È Quindi Un Insieme di Elementi con Significato Complesso

SINTASSI DEL COSTRUTTO $\Rightarrow$:= che non si CONFONDE
con = usato per l'UGUAGLIANZA